

PROJETO CHAMADO URBANO+

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP — 1º BIMESTRE)

Instituição: Unicesumar (Maringá - PR) Curso: Engenharia de Software — 4º Semestre (2026.2)
ODS 11 — Cidades e Comunidades Sustentáveis

Equipe: Eduardo Henrique de Souza da Silva Rafael Cescate do Carmo Cauê Licce

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

Dona Marlene mora no bairro Jardim das Palmeiras e já faz um tempo que tem um poste de luz apagado bem na esquina da casa dela. Quando ela tenta resolver isso, liga pro telefone geral da prefeitura, fica sendo passada de setor em setor e no final ninguém dá retorno se o problema foi resolvido ou não. Do outro lado, o agente Carlos recebe reclamação por telefone e WhatsApp, sem nenhum lugar pra anotar isso direito, e acaba perdendo chamado ou recebendo o mesmo problema duas vezes.

Esse projeto tem tudo a ver com a ODS 11 da ONU (Cidades e Comunidades Sustentáveis). Se a gente centralizar os chamados de manutenção da cidade, a prefeitura consegue ver melhor o que está acontecendo, resolve mais rápido e o morador passa a confiar mais no canal oficial.

Resumo do Discovery (Jornada do Usuário e Matriz CSD)

Pensando na jornada do usuário, a gente percebeu que o maior problema do morador não é só abrir o chamado, é não saber o que aconteceu depois — isso faz ele ligar de novo e duplicar o chamado. Na Matriz CSD (Certezas, Suposições e Dúvidas), ficou claro que uma coisa é certa: o sistema precisa mostrar o status do chamado (Aberto → Em Andamento → Resolvido) tanto pro morador quanto pro agente, pra evitar esse retrabalho.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

RF01 — Cadastrar moradores, com nome, e-mail, endereço e telefone.

RF02 — Abrir um chamado, informando tipo de problema, descrição, localização e data.

RF03 — Classificar o chamado em um dos três tipos — Infraestrutura, Iluminação ou Limpeza Urbana — cada um com sua própria regra de prioridade.

RF04 — Cadastrar um agente municipal, que fica responsável por um ou mais chamados.

RF05 — Atualizar o status do chamado (Aberto → Em Andamento → Resolvido), guardando o histórico de mudanças.

RF06 — Listar todos os chamados de um morador específico.

RF07 — Cancelar (excluir) um chamado, só pelo morador que abriu ele.

Por que esse escopo? Cada requisito já diz de forma direta o que o sistema vai fazer pelo usuário, sem encher de termo técnico — e o RF03 já deixa preparado o terreno pra herança e polimorfismo que vamos usar na Entrega 2.

3. O PLANEJAMENTO ÁGIL (CRONOGRAMA / BACKLOG)

Abaixo está como a gente pensa em dividir o desenvolvimento do 2º bimestre, em Sprints com histórias de usuário (User Stories).

Sprint / Data	Épico	User Story	Responsável
Sprint 1 05/09 - 19/09	Cadastro e Autenticação	COMO UM morador EU QUERO me cadastrar no sistema PARA QUE eu possa abrir chamados de manutenção urbana.	Cauê Licce
Sprint 2 20/09 - 04/10	Abertura de Chamados	COMO UM morador EU QUERO abrir um chamado especificando tipo e localização PARA QUE o problema seja registrado e encaminhado ao setor responsável.	Eduardo H. de Souza da Silva

Sprint / Data	Épico	User Story	Responsável
Sprint 3 05/10 - 19/10	Gestão de Status	COMO UM agente municipal EU QUERO atualizar o status de um chamado PARA QUE o morador acompanhe o andamento da resolução.	Rafael Cescate do Carmo
Sprint 4 20/10 - 03/11	Persistência e Relatórios	COMO UM agente municipal EU QUERO consultar o histórico de chamados por status PARA QUE eu possa priorizar as demandas mais antigas.	Squad completo

As datas seguem o calendário previsto pra instituição no 2º bimestre, mas vale conferir se não mudou nada.

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

Pra deixar o código organizado e fácil de mexer depois, o sistema vai ser feito em **Java**. A gente escolheu Java porque é fortemente tipado e tem tudo que precisamos de Orientação a Objetos: o problema pede regras bem definidas (cálculo de prioridade por tipo de chamado, permissões diferentes pra morador e agente), e usando bem abstração, encapsulamento, herança e polimorfismo o código fica mais fácil de manter.

Pro banco de dados, vamos usar **MySQL**, organizado em camadas (Model → DAO → Service). O MySQL garante que os dados fiquem consistentes entre Morador, AgenteMunicipal e Chamado, e separar em camadas ajuda a testar e mudar o sistema sem bagunçar tudo depois.

Como no curso de Engenharia de Software a gente ainda não viu interface gráfica, o sistema vai rodar direto no terminal — o foco é a arquitetura por trás, não a aparência.

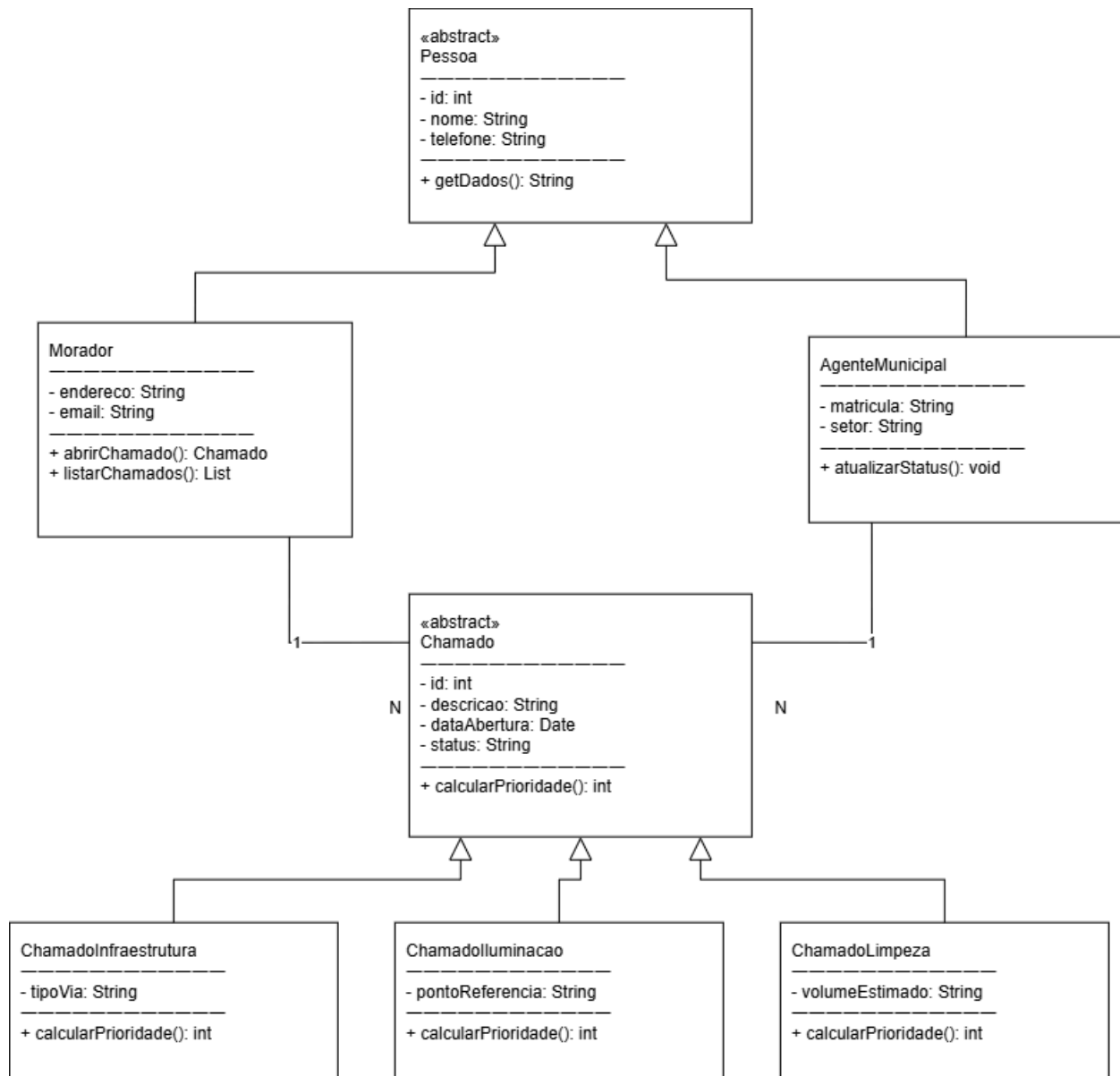
5. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

5.1 Link do Repositório

Repositório Público GitHub: <https://github.com/CaueLicce/Entrega-AEP>

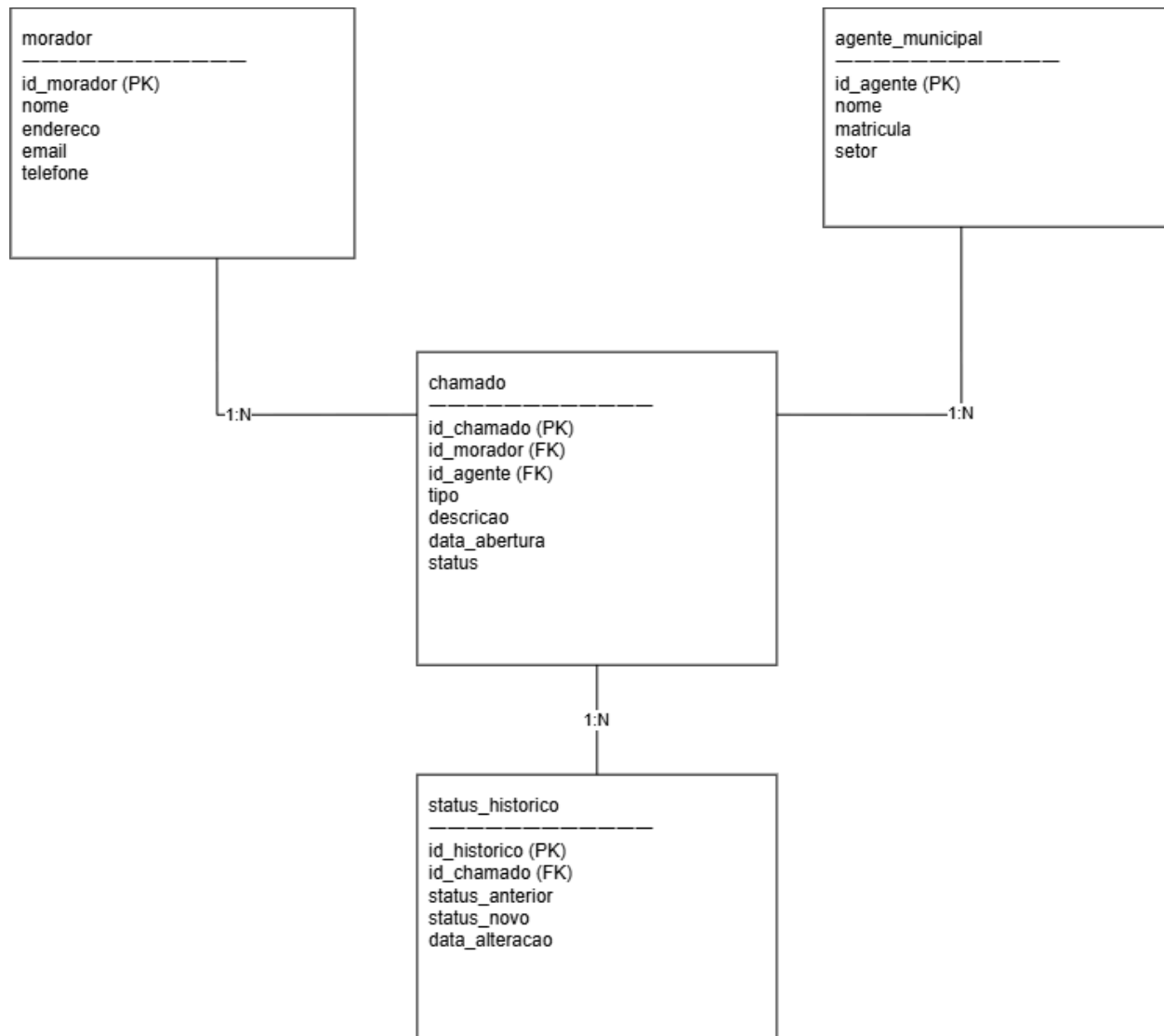
O repositório já está com as pastas `/src`, `/docs` e `/database` criadas, prontas pra receber o código no 2º bimestre.

5.2 Diagrama de Classes (UML)



- **Herança:** criamos uma superclasse **Pessoa**, abstrata, com os dados que **Morador** e **AgenteMunicipal** têm em comum (id, nome, telefone). Assim não precisa repetir esses campos em cada classe.
- **Composição 1:N:** um **Morador** pode abrir vários **Chamado**, e um **AgenteMunicipal** pode cuidar de vários **Chamado** ao mesmo tempo.
- **Polimorfismo:** **Chamado** também é abstrata e tem o método `calcularPrioridade()`. Cada subclasse (**ChamadoInfraestrutura**, **ChamadoIluminacao**, **ChamadoLimpeza**) sobrescreve esse método (`@Override`) do seu jeito, mudando como a prioridade é calculada.

5.3 Diagrama de Banco de Dados (DER)



- **Tabela morador**: guarda os dados de quem abre os chamados (nome, endereço, e-mail, telefone).
- **Tabela agente_municipal**: guarda os dados de quem cuida e atualiza os chamados (nome, matrícula, setor).
- **Tabela chamado**: tem as chaves estrangeiras **id_morador** e **id_agente**, além do **tipo** (pra saber qual subclasse é) e do **status** atual.
- **Tabela status_historico**: guarda o registro de cada mudança de status do chamado (**status_anterior**, **status_novo**, **data_alteracao**) — é o que garante o RF05.